C++ Object and Class

Since C++ is an object-oriented language, programs are designed using objects and classes in C++.

C++ Object

In C++, Object is a real-world entity, for example, chair, car, pen, mobile, laptop etc.

In other words, an object is an entity that has state and behavior. Here, state means data and behavior means functionality. Object is a runtime entity; it is created at runtime. Object is an instance of a class. All the members of the class can be accessed through object.

Let's see an example to create object of student class using s1 as the reference variable.

```
Student s1;  //creating an object of Student
```

In this example, Student is the type and s1 is the reference variable that refers to the instance of Student class.

C++ Class

In C++, class is a group of similar objects. It is a template from which objects are created. It can have fields, methods, constructors etc.

Let's see an example of C++ class that has three fields only.

```
class Student
 {
    public:
    int id;  //field or data member
    float salary; //field or data member
    String name;//field or data member
 }
```

Let's see an example of class that has two fields: id and name. It creates instance of the class, initializes the object and prints the object value.

```
#include <iostream>
using namespace std;
class Student {
  public:
    int id;//data member (also instance variable)
    string name;//data member(also instance variable)
};
int main() {
   Student s1; //creating an object of Student
   s1.id = 201;
   s1.name = "Sonoo Jaiswal";
   cout<<s1.id<<endl;
   cout<<s1.name<<endl;
   return 0;
}
```

Let's see another example of C++ class where we are initializing and displaying object through method.

```
#include <iostream>
using namespace std;
class Student {
  public:
     int id;//data member (also instance variable)
     string name;//data member(also instance variable)
     void insert(int i, string n)
      {
         id = i;
         name = n;
      }
     void display()
      {
         cout<<id<<"  "<<name<<endl;
      }
};
int main(void) {
   Student s1; //creating an object of Student
   Student s2; //creating an object of Student
   s1.insert(201, "Sonoo");
   s2.insert(202, "Nakul");
   s1.display();
   s2.display();
   return 0;
}
```

C++ Constructor

A particular procedure called a constructor is called automatically when an object is created in C++. In general, it is employed to create the data members of new things. In C++, the class or structure name also serves as the constructor's name. When an object is completed, the constructor is called. Because it creates the values or gives data for the thing, it is known as a constructor.

The Constructors prototype looks like this:
<class-name> (list-of-parameters);

The following syntax is used to define the class's constructor:
<class-name> (list-of-parameters) { // constructor definition }

The following syntax is used to define a constructor outside of a class:
<class-name>: :<class-name> (list-of-parameters){ // constructor definition}

Constructors lack a return type since they don't have a return value.
There can be two types of constructors in C++.

- Default constructor
- Parameterized constructor

A constructor which has no argument is known as the default constructor. It is invoked at the time of creating object.
Let's see the simple example of C++ default Constructor.

```
#include <iostream>
using namespace std;
class Employee
 {
   public:
       Employee()
       {
           cout<<"Default Constructor Invoked"<<endl;
       }
};
int main(void)
{
    Employee e1; //creating an object of Employee
    Employee e2;
    return 0;
}
```

C++ Parameterized Constructor

A constructor which has parameters is called parameterized constructor. It is used to provide different values to distinct objects.

Let's see the simple example of C++ Parameterized Constructor.

```
#include <iostream>
using namespace std;
class Employee {
  public:
     int id;//data member (also instance variable)
     string name;//data member(also instance variable)
     float salary;
     Employee(int i, string n, float s)
      {
         id = i;
         name = n;
         salary = s;
      }
     void display()
      {
         cout<<id<<"  "<<name<<"  "<<salary<<endl;
      }
};
int main(void) {
   Employee e1 =Employee(101, "Sonoo", 890000); //creating an object of Employee
   Employee e2=Employee(102, "Nakul", 59000);
   e1.display();
   e2.display();
   return 0;
}
```

What distinguishes constructors from a typical member function?

- Constructor's name is the same as the class's
- Default There isn't an input argument for constructors. However, input arguments are available for copy and parameterized constructors.
- There is no return type for constructors.
- An object's constructor is invoked automatically upon creation.
- It must be shown in the classroom's open area.
- The C++ compiler creates a default constructor for the object if a constructor is not specified (expects any parameters and has an empty body).

What are the characteristics of a constructor?

- The constructor has the same name as the class it belongs to.
- Although it is possible, constructors are typically declared in the class's public section. However, this is not a must.
- Because constructors don't return values, they lack a return type.
- When we create a class object, the constructor is immediately invoked.
- Overloaded constructors are possible.
- Declaring a constructor virtual is not permitted.
- One cannot inherit a constructor.
- Constructor addresses cannot be referenced to.
- When allocating memory, the constructor makes implicit calls to the new and delete operators.

C++ Destructor

A destructor works opposite to constructor; it destructs the objects of classes. It can be defined only once in a class. Like constructors, it is invoked automatically.

A destructor is defined as constructor. It must have the same name as class. But it is prefixed with a tilde sign (~).

C++ this Pointer

In C++ programming, this is a keyword that refers to the current instance of the class.

There can be 3 main usages of this keyword in C++.

- It can be used to pass current object as a parameter to another method.
- It can be used to refer to current class instance variable.
- It can be used to declare indexers.

```
Employee(int id, string name, float salary)
{
    this->id = id;
    this->name = name;
    this->salary = salary;
}
```

C++ static

In C++, static is a keyword or modifier that belongs to the type not instance. So instance is not required to access the static members. In C++, static can be field, method, constructor, class, properties, operator and event.

Advantage of C++ static keyword

Memory efficient: Now we don't need to create instances for accessing the static members, so it saves memory. Moreover, it belongs to the type, so it will not get memory each time when instance is created.

C++ Static Field

A field which is declared static is called static field. Unlike instance field which gets memory each time whenever you create object, there is only one copy of static field created in the memory. It is shared with all the objects.

C++ Structs

In C++, classes and structs are blueprints that are used to create the instance of a class. Structs are used for lightweight objects such as Rectangle, color, Point, etc.

Unlike class, structs in C++ are value type than reference type. It is useful if you have data that is not intended to be modified after creation of struct.

C++ Structure is a collection of different data types. It is similar to the class that holds different types of data.

The Syntax of Structure

```
struct structure_name
{
    // member declarations.
}
```

```
struct Student
{
   char name[20];
    int id;
    int age;
}
```

Let's see a simple example of struct Rectangle which has two data members width and height.

```
#include <iostream>
using namespace std;
 struct Rectangle
{
   int width, height;

 };
int main(void) {
    struct Rectangle rec;
    rec.width=8;
    rec.height=5;
   cout<<"Area of Rectangle is: "<<(rec.width * rec.height)<<endl;
 return 0;
}
```

Structure v/s Class

Structure	Class
If access specifier is not declared explicitly, then by default access specifier will be public.	If access specifier is not declared explicitly, then by default access specifier will be private.

Syntax of Structure:
 	
struct structure_name
{
// body of the structure.
}
	Syntax of Class:
	
	class class_name
	{
	// body of the class.
	}

The instance of the structure is known as "Structure variable".	The instance of the class is known as "Object of the class".

C++ Enumeration

Enum in C++ is a data type that contains fixed set of constants.

It can be used for days of the week (SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY and SATURDAY) , directions (NORTH, SOUTH, EAST and WEST) etc. The C++ enum constants are static and final implicitly.

C++ Enums can be thought of as classes that have fixed set of constants.

Points to remember for C++ Enum

- Enum improves type safety
- Enum can be easily used in switch
- Enum can be traversed
- Enum can have fields, constructors and methods
- Enum may implement many interfaces but cannot extend any class because it internally extends Enum class

Let's see the simple example of enum data type used in C++ program.

```
#include <iostream>
using namespace std;
enum week { Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday };
int main()
{
   week day;
   day = Friday;
   cout << "Day: " << day+1<<endl;
   return 0;
}
```

C++ Friend function

If a function is defined as a friend function in C++, then the protected and private data of a class can be accessed using the function. By using the keyword friend compiler knows the given function is a friend function. For accessing the data, the declaration of a friend function should be done inside the body of a class starting with the keyword friend.

Declaration of friend function in C++

```
class class_name
{
   friend data_type function_name(argument/s);          // syntax of friend function.
};
```

Characteristics of a Friend function:

- The function is not in the scope of the class to which it has been declared as a friend.
- It cannot be called using the object as it is not in the scope of that class.
- It can be invoked like a normal function without using the object.
- It cannot access the member names directly and has to use an object name and dot membership operator with the member name.
- It can be declared either in the private or the public part.

Let's see the simple example of C++ friend function used to print the length of a box.

```
#include <iostream>
using namespace std;
class Box
{
   private:
      int length;
   public:
      Box(): length(0) { }
      friend int printLength(Box); //friend function
};
int printLength(Box b)
{
  b.length += 10;
   return b.length;
}
int main()
{
   Box b;
   cout<<"Length of box: "<< printLength(b)<<endl;
   return 0;
}
```

C++ Friend class

A friend class can access both private and protected members of the class in which it has been declared as friend.

Let's see a simple example of a friend class.

```
#include <iostream>

using namespace std;

class A
{
   int x =5;
   friend class B;          // friend class.
};
class B
{
  public:
   void display(A &a)
   {
      cout<<"value of x is : "<<a.x;
   }
};
int main()
{
   A a;
   B b;
   b.display(a);
   return 0;
}
```

Output:
value of x is : 5

In the above example, class B is declared as a friend inside class A. Therefore, B is a friend of class A. Class B can access the private members of class A.